

SAE Capteur hangar de patates



SOMMAIRE

Cahier des charges.....	3
Contraintes techniques.....	3
Contraintes de performance.....	3
Architecture générale du système.....	4
ESP32.....	4
Capteur de luminosité.....	6
Capteur de température et d'humidité.....	7
Capteur de qualité de l'air.....	8
Capteur d'éthylène.....	9
Câblage.....	11
Schéma électronique.....	12
Circuit électrique sur breadboard.....	12
Circuit Imprimé (PCB).....	13
Visualisation 3D.....	13
CODE.....	14
Traitement des Données et Algorithmes de Conversion.....	16
1. Luminosité (Conversion Analogique-Numérique).....	16
2. Éthylène (Analyse de Chaîne de Caractères / Parsing).....	16
3. Qualité de l'Air (Compensation Environnementale).....	17
4. Température et Humidité (Décodage I2C).....	17

L'objectif du projet est de concevoir un système autonome capable de surveiller en continu les paramètres atmosphériques d'un hangar de stockage de pommes de terre.

Les données doivent être fiables, la maintenance minimale et la communication entièrement sans fil afin de couvrir plusieurs milliers de mètres carrés.

Cahier des charges

Le système doit être capable de mesurer :

Grandeur	Plage	Rôle dans le hangar
Température	0 à 15 °C	Prévenir germination / dégradation
Humidité relative	60 à 90 %	Éviter moisissures ou déshydratation
Luminosité	Lumière / pas de lumière	Détection d'intrusion lumineuse (portes ouvertes, panne éclairage)
CO ₂ (équivalent)	300 à 2000 ppm	Indicateur du renouvellement d'air et de la respiration des tubercules
Éthylène	0 à 40 ppm	Indicateur de stress, maturation, germination

Contraintes techniques

- Communication sans fil obligatoire.
- Le hangar a une surface de plusieurs milliers de m² → nécessité d'un réseau maillé.
- Les capteurs doivent être alimentés par pile → faible consommation indispensable.
- Les composants doivent résister aux conditions du hangar (froid, humidité, poussière).
- Maintenance réduite : autonomie > 6 mois, calibrations rares.
- Coût réduit pour permettre la multiplication des nœuds capteurs.

Contraintes de performance

- Fiabilité de la mesure ± 10 % sur le CO₂ / TVOC.
- Précision température ± 0.3 °C, humidité ± 2 %.
- Temps de mise à jour des données ≤ 2 s.
- Décodage fiable des trames UART du capteur d'éthylène.
- Conversion analogique 12 bits pour la luminosité.

Architecture générale du système

Le système est composé de plusieurs nœuds capteurs répartis dans le hangar.

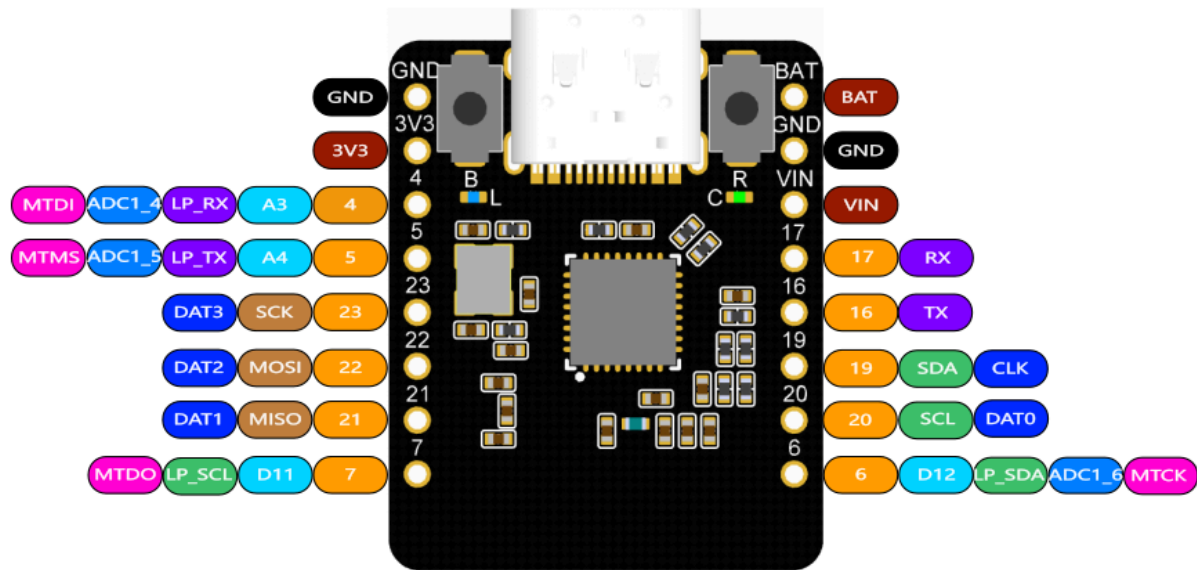
Chaque nœud contient :

- Un microcontrôleur ESP32-C6,
- Les capteurs (T°/HR, luminosité, qualité de l'air, éthylène),
- Un module radio Zigbee intégré,
- Une alimentation par pile.

Un coordinateur Zigbee peut centraliser toutes les données sur une interface PC ou un serveur.

Nous avons utilisé plusieurs composants afin de répondre à la problématique de ce projet, voici la liste des composants :

ESP32



Pin Definition

- Power: Broche d'alimentation
 - VIN: 5V DC (Courant Continu)
 - 3V3: Sortie stable de 3.3V
- GND: Broche de masse commune (ground)
- GPIO: Numéro GPIO par défaut de l'ESP32
- Arduino: Correspondance des GPIO du Beetle ESP32-C6 dans Arduino
- ADC: Broche de conversion analogique-numérique par défaut de l'ESP32
- I2C: Interface I2C
 - Beetle ESP32-C6 I2C mapping in Arduino
 - LP_SDA/SCL: Low power I2C pin
- UART: Interface UART
 - LP_TX/RX: Low power UART pin
- SPI: Correspondance SPI du Beetle ESP32-C6 dans Arduino
- SDIO: Broche SDIO par défaut de l'ESP32
- JTAG: Interface de débogage (debug)

L'ESP32-C6 peut descendre à 20–50 μ A en deep-sleep, ce qui est un point clé pour un système sur piles.

Capteur de luminosité

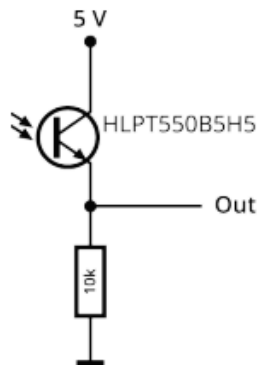
PHOTOTRANSISTOR

in a compact design



The SEN-KY054 phototransistor is a compact and reliable NPN phototransistor.

It offers fast response times and low susceptibility to interference, making it ideal for applications such as ambient light monitoring, optical encoders and safety solutions. Its design allows easy integration into space-critical systems, and it operates reliably at temperatures from -25°C to +70°C - an ideal choice for precise light detection.



Published: 28.10.2024

MAIN FEATURES

Model	Phototransistor
Chipset	HLPT550B5H5
Sensor type	analog brightness output
Compatible with	Arduino, Raspberry Pi, Raspberry Pi Pico, micro:bit and more

FURTHER SPECIFICATIONS

Supply voltage	1.8 - 5 V
Output voltage	$\leq$ Supply voltage
Fixed known resistance	10 k Ω
Operating temperature	-25 to 70 °C

FURTHER INFORMATION

Dimensions	28 x 15 x 7 mm
Weight	2 g
Item number	SEN-KY054
EAN	4250236828398
Customs tariff number	90278910

Nom du capteur : SEN-KY054

Alimentation : 1.8V à 5V (Compatible avec le 3.3V de l'ESP32).

consommation : ~10 μ A

Type de sortie : Signal Analogique (La tension varie selon l'intensité lumineuse).

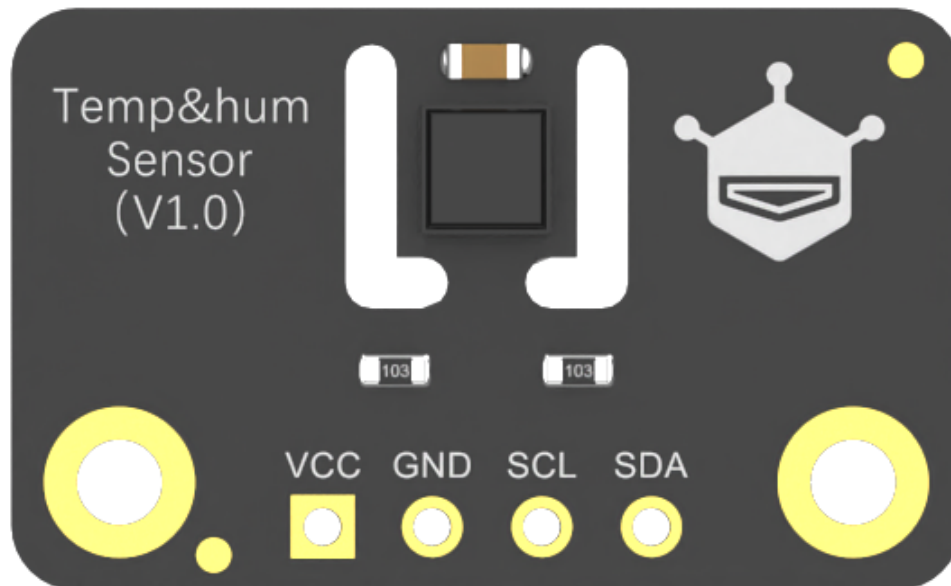
Résistance intégrée : 10 k Ω (Résistance fixe pour le pont diviseur de tension).

Température de fonctionnement : -25°C à +70°C.

Durée de mise en route: 0s (immédiat)

Durée de vie : 10-20 ans

Capteur de température et d'humidité



Pin	Function	Description
VCC	+	Power Supply: DC 2V~5V
GND	-	--
SCL	I2C Clock Line	--
SDA	I2C Data Line	--

Nom du capteur : AHT 20

Alimentation : 2.0V à 5.5V (Idéal 3.3V)

consommation : 0.25 mA en mesure

Plage Température : -40°C à +85°C

Précision Température : ±0.3 °C

Plage Humidité : 0% à 100% RH

Précision Humidité : ±2% RH

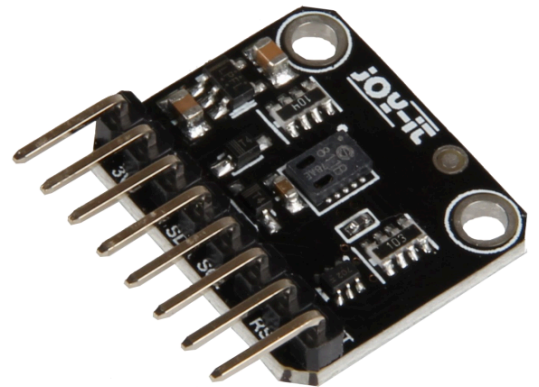
Durée de mise en route: 5 à 30 secondes

Durée de vie : environ 10 ans

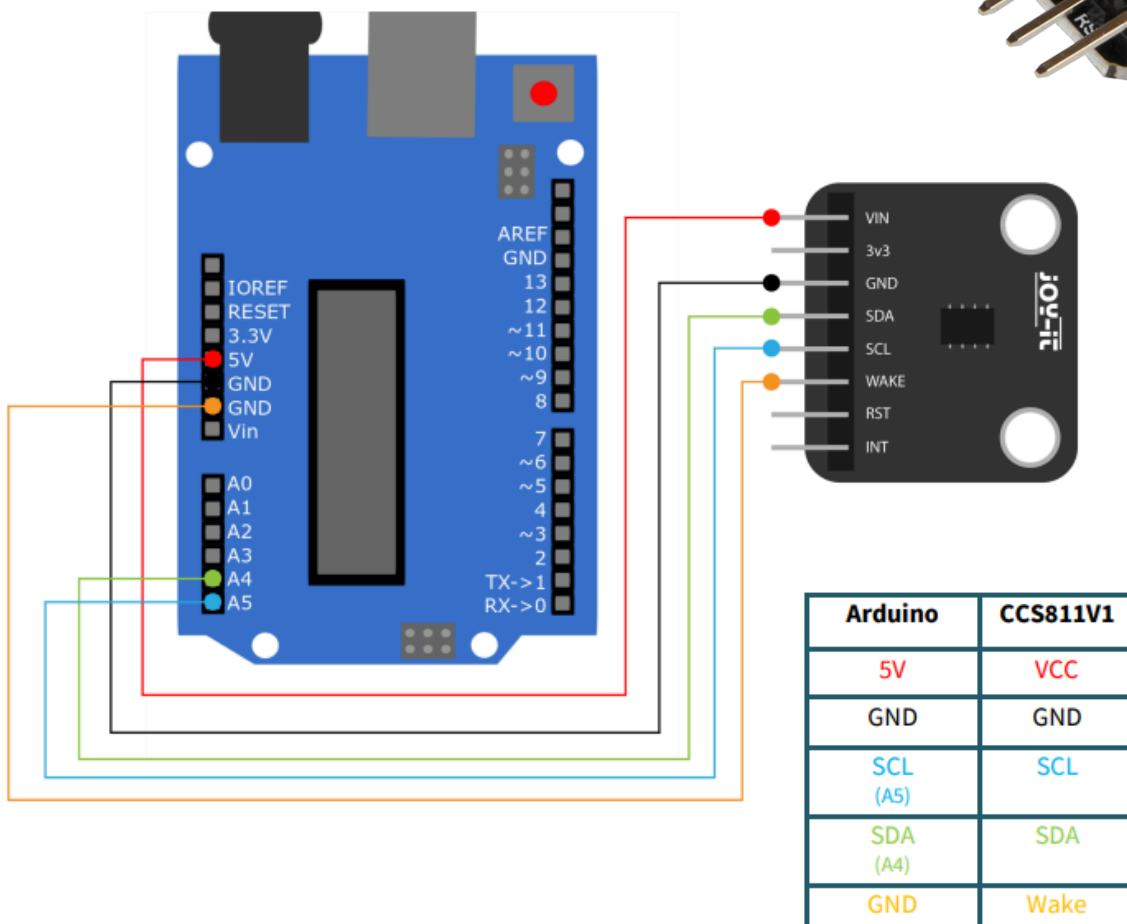
Capteur de qualité de l'air

AIR QUALITY SENSOR

SEN-CCS811V1



1. Connection



Nom du capteur: CCS811V1

Alimentation : 1.8V à 3.6V (Attention : 3.3V recommandé)

consommation : 0.6 à 1 mA selon mode

Plage eCO2 : 400 ppm à 8192 ppm

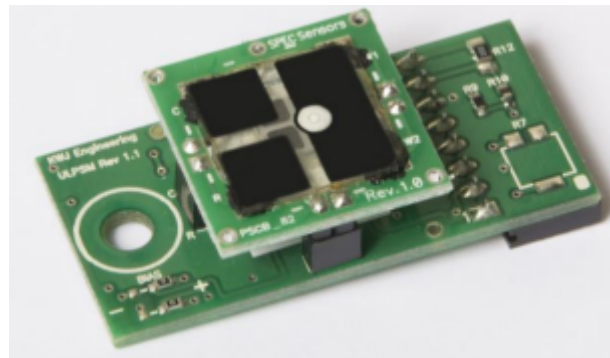
Plage TVOC : 0 ppb à 1187 ppb

Durée de mise en route : 20 minutes pour être précis, 48h pour une calibration parfaite.

Temps de rodage (Burn-in) : 48 heures (Si le capteur est neuf).

Durée de vie : > 5 ans

Capteur d'éthylène



MEASUREMENT PERFORMANCE CHARACTERISTICS

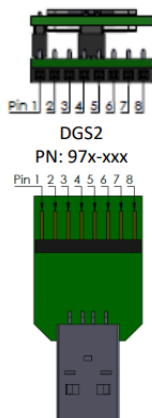
At Standard Conditions: 25 °C, 50% RH and 1 atm	
Measurement Accuracy [1,4]	± 15%
Measurement Repeatability	< ± 3% or <i>DGS Zero Accuracy</i> , whichever is greater
Response Time [5]	< 15 seconds
Power Consumption	1.32 mW in sleep mode 33 mW in active mode
Expected Operating Life [6]	> 5 years (10 years @ 25 ± 10 °C; 60 ± 30% RH)
Operating Temperature Range [1,5]	-20 to 40 °C (-30 to 55 °C intermittent)
Operating Humidity Range	15 to 95% (0 to 100% non-condensing intermittent)
Mechanical Dimensions	1.75 x 0.82 x 0.35 in. (44.5 x 20.8 x 8.9 mm)
Weight	< 2 Ounces

ABSOLUTE MAXIMUM SPECIFICATIONS

Parameter	Conditions	Min.	Typ.	Max.	Units
Maximum Concentration	Short term exposure			120	% Range
Storage Temperature	Vapor sealed @ 50% RH	5	20	30	°C
Storage Humidity	Non-condensing, vapor sealed	20	50	80	% RH
Storage Pressure	Vapor sealed	0.8	1	1.2	atm
Storage Time	Vapor sealed	-	12	-	Months
Operating Temperature	< 10 hours	-40	-	50	°C
Operating Humidity	< 10 hours, non-condensing	0	-	100	% RH
Operating Pressure	Transient pulses	-.5	0	.5	atm
Operating Temperature	Continuous	-30	25	50	°C
Operating Humidity	Continuous, non-condensing	10	50	95	% RH
Operating Pressure	Continuous	0.8	1	1.2	atm
ESD Rating	Human Body Model	2		8	kV

ELECTRICAL SPECIFICATIONS

Parameter	Conditions	Min.	Typ.	Max.	Units
Supply Voltage, V+		2.6	3.3	3.6	V
Supply Current	V+ = 3.3 V, Active Mode V+ = 3.3 V, Sleep Mode		10 0.4		mA
Power Consumption	V+ = 3.3 V, Active Mode V+ = 3.3 V, Sleep Mode		33 1.32		mW



USB-to-UART Adapter
PN: 981-035

DGS2 Pinout		
Pin #	Function	Description
1	N/C	
2	RXD	DGS2 UART Receive
3	TXD	DGS2 UART Transmit
4	N/C	
5	N/C	
6	GND	Ground
7	N/C	
8	V+	2.6 to 3.6 V

	ASCII character command	Name	Description
Operate	\r [Enter]	Single Measurement	A measurement string is returned. (See Example Measurement String)
	C (uppercase)	Continuous Measurement	Toggles on/off continuous measurement. A measurement string is returned approximately once per second. (See Example Measurement String)
	e	EEPROM Settings	Settings and diagnostic info are returned. (See Example EEPROM Settings Output)
	r	Reset/Restart	Module reset. EEPROM Settings are retained.

Nom du capteur : DGS2 970-650

Alimentation : 2.6V à 3.6V (Idéal 3.3V)

consommation : ~10 mA en fonctionnement

Plage de mesure : 0 à 40 ppm (Certains modèles vont jusqu'à 100 ppm)

Résolution : 0.02 ppm (20 ppb)

Précision : ±15 % de la lecture

Durée De mise en route: 1-24 heures

Durée de vie : > 5 ans (Souvent 10 ans dans de bonnes conditions)

Câblage

1. Capteur Température/Humidité (AHT20)

- Pin **SDA** : Pin **19** de l'ESP.
- Pin **SCL** : Pin **20** de l'ESP.
- *(Partage les mêmes pins que le CCS811)*

2. Capteur Qualité d'Air (CCS811)

- Pin **SDA** : Pin **19** de l'ESP.
- Pin **SCL** : Pin **20** de l'ESP.
- Pin **WAKE: GND** (Pour l'activer).

3. Capteur Éthylène (DGS2)

- Pin **RX** du capteur: Pin **16 (TX)** de l'ESP.
- Pin **TX** du capteur : Pin **17 (RX)** de l'ESP.
- *(Attention : On croise toujours, le TX va sur le RX).*

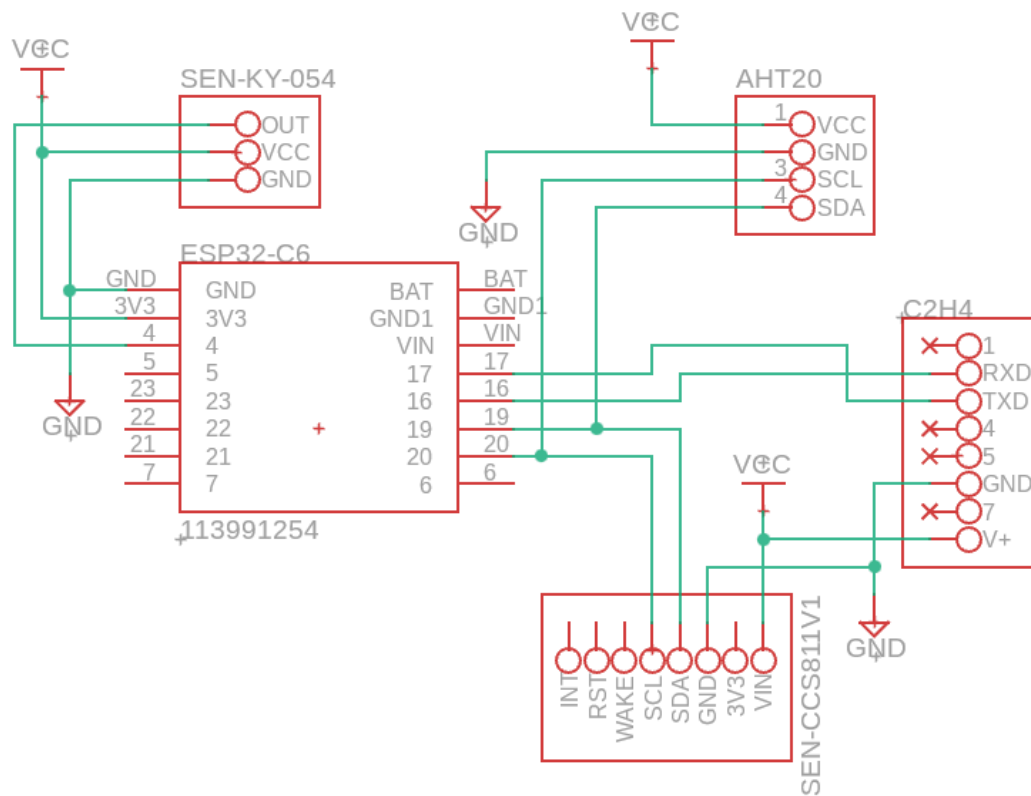
4. Capteur Lumière (LDR)

- Pin **Signal** : Pin **4** de l'ESP.

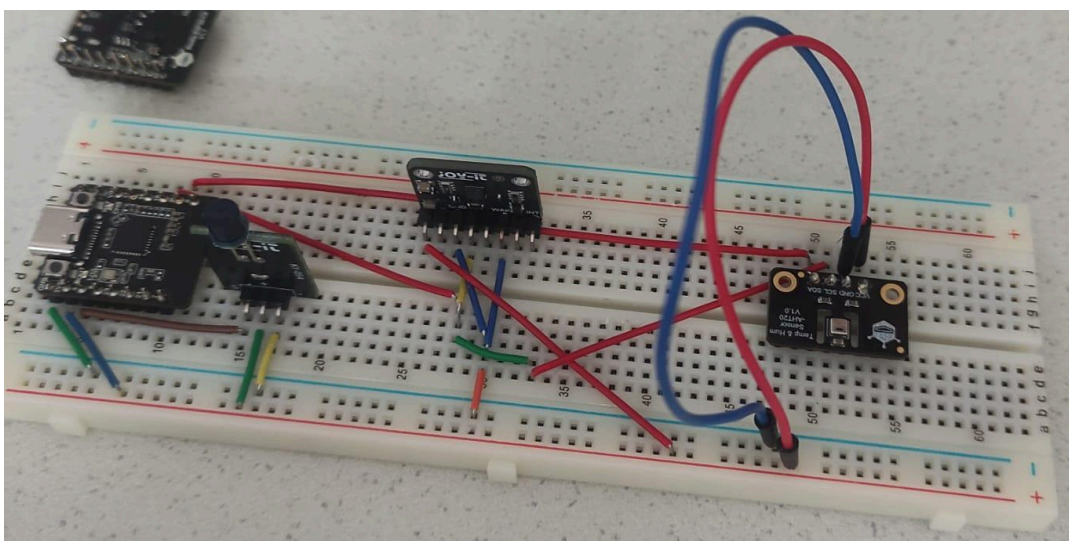
Alimentation (Pour tous les capteurs)

- **VCC** : **3V3** de l'ESP.
- **GND** : **GND** de l'ESP.

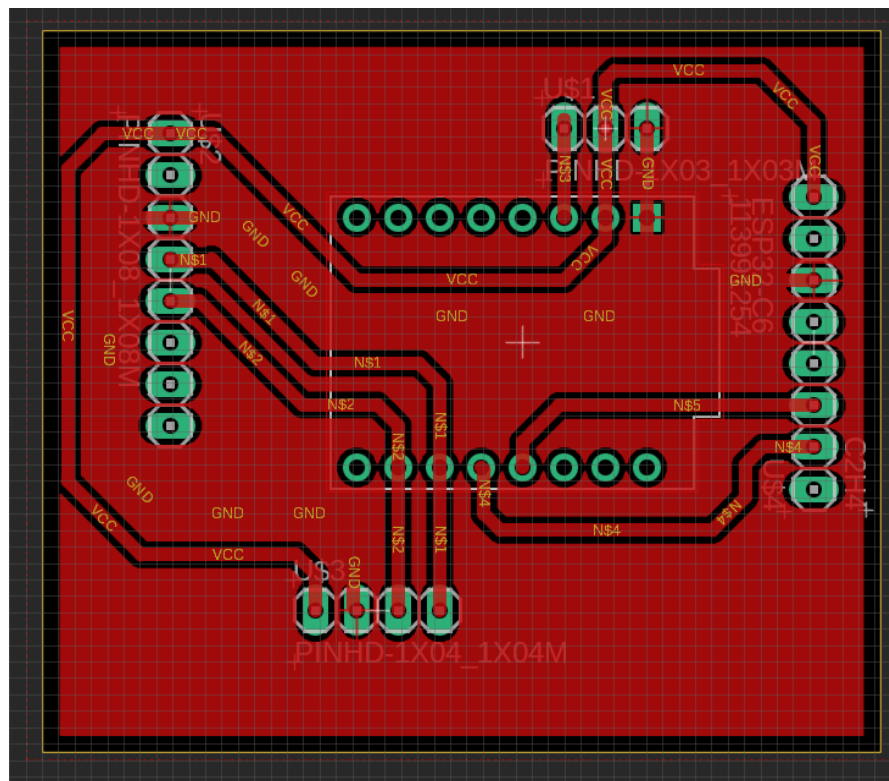
Schéma électronique



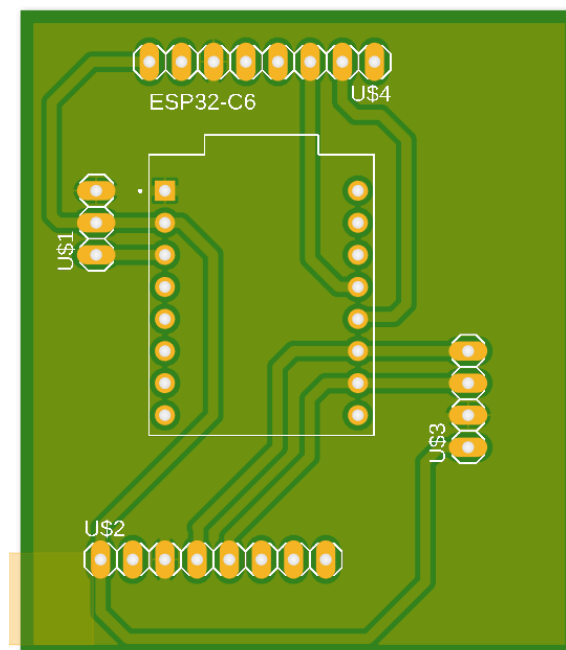
Circuit électrique sur breadboard



Circuit Imprimé (PCB)



Visualisation 3D



CODE

```
#include <Wire.h>
#include "DFRobot_AHT20.h"
#include "Adafruit_CCS811.h"

#define PIN_LUMIERE 4

DFRobot_AHT20 capteurTempHum;
Adafruit_CCS811 capteurCO2;

float temperature = 0;
float humidite = 0;
float co2 = 0;
float voc = 0;
float lux = 0;
float ethylenePPM = 0;

void setup() {
  Serial.begin(115200);
  Serial1.begin(9600, SERIAL_8N1, 17, 16);

  pinMode(PIN_LUMIERE, INPUT);
  analogReadResolution(12);

  Wire.begin(19, 20);

  capteurTempHum.begin();

  if (!capteurCO2.begin()) {
    Serial.println("Erreur CCS811");
  } else {
    capteurCO2.setDriveMode(CCS811_DRIVE_MODE_1SEC);
  }

  delay(1000);
}

void loop() {
  if (capteurTempHum.startMeasurementReady(true)) {
    temperature = capteurTempHum.getTemperature_C();
    humidite = capteurTempHum.getHumidity_RH();
  }

  capteurCO2.setEnvironmentalData(humidite, temperature);

  if (capteurCO2.available()) {
```

```

    if (!capteurCO2.readData()) {
        co2 = capteurCO2.getCO2();
        voc = capteurCO2.getTVOC();
    }
}

int lecture = analogRead(PIN_LUMIERE);
lux = (lecture / 4095.0) * 677.6;

while (Serial1.available()) { Serial1.read(); }

Serial1.print("\r");

long debut = millis();
while (!Serial1.available() && millis() - debut < 1000) {
    delay(10);
}

if (Serial1.available()) {
    String reponse = Serial1.readStringUntil('\n');

    int v1 = reponse.indexOf(',');
    int v2 = reponse.indexOf(',', v1 + 1);

    if (v1 > 0 && v2 > 0) {
        String strGaz = reponse.substring(v1 + 1, v2);
        ethylenePPM = strGaz.toFloat() / 1000.0;
    }
}

Serial.println("--- MONITORING ---");
Serial.print("Temp    : "); Serial.print(temperature); Serial.println(" C");
Serial.print("Humidite : "); Serial.print(humidite);   Serial.println(" %");
Serial.print("Lumiere  : "); Serial.print(lux);        Serial.println(" Lux");
Serial.print("CO2     : "); Serial.print(co2);         Serial.println(" ppm");
Serial.print("COV     : "); Serial.print(voc);         Serial.println(" ppb");
Serial.print("Ethylene : "); Serial.print(ethylenePPM); Serial.println(" ppm");
Serial.println("-----");
Serial.println("");

delay(2000);
}

```

Traitement des Données et Algorithmes de Conversion

Le microcontrôleur ESP32-C6 agit comme une unité de conversion centrale. Il reçoit des données brutes de nature différente (tension analogique, texte ASCII, bits I2C) et leur applique des transformations mathématiques ou logiques pour obtenir des mesures physiques.

1. Luminosité (Conversion Analogique-Numérique)

Le capteur (Phototransistor) retourne une tension électrique proportionnelle à la lumière. L'ESP32 ne lit pas des volts, mais une valeur numérique brute.

- **Acquisition** : Le convertisseur analogique-numérique (ADC) de l'ESP32 échantillonne la tension sur 12 bits, donnant un nombre entier entre **0** (0V, noir complet) et **4095** (3.3V, saturation lumineuse).
- **Lissage** : Une moyenne arithmétique est calculée sur 20 lectures successives pour éliminer le bruit électrique.
- Calcul de transformation :

$$\text{Luminosité(Lux)} = \left(\frac{\text{Valeur}_{ADC}}{4095} \right) \times \text{Facteur}_{Calibration}$$

(Dans notre code, le facteur de calibration est fixé à 677.6).

2. Éthylène (Analyse de Chaîne de Caractères / Parsing)

Le capteur DGS2 est un module intelligent qui envoie les données sous forme de texte (protocole UART).

- **Acquisition** : L'ESP32 envoie une commande de déclenchement (r). Le capteur répond par une chaîne de caractères complexe contenant plusieurs informations séparées par des virgules (ex: "SN, 1500, 24, 45").
- **Extraction** : L'algorithme recherche la position des virgules dans le texte pour isoler la sous-chaîne correspondant à la concentration de gaz (ici "1500").
- Conversion d'unité : La valeur extraite est en PPB (Parties Par Milliard). Pour obtenir des PPM (Parties Par Million), une division est appliquée :

$$\text{Concentration(PPM)} = \frac{\text{Valeur}_{PPB}}{1000}$$

3. Qualité de l'Air (Compensation Environnementale)

Le capteur CCS811 mesure la résistance d'oxydes métalliques (MOX) pour estimer le CO2 et les COV.

- **Acquisition Croisée** : Ce capteur est sensible à la température et l'humidité. Avant de lire la qualité de l'air, le code injecte les valeurs réelles mesurées par le capteur AHT20 dans les registres du CCS811.
- **Calcul Interne** : Le capteur utilise ces données climatiques pour compenser sa propre dérive et calculer une valeur corrigée (eCO2 et TVOC) que l'ESP32 vient ensuite lire via le bus I2C.

4. Température et Humidité (Décodage I2C)

Le capteur AHT20 envoie des trames de données binaires via le bus I2C.

- **Traitement** : La bibliothèque dédiée (DFRobot_AHT20) interroge le capteur, récupère les octets bruts, et applique les formules de conversion du constructeur pour transformer ces bits en degrés Celsius (°C) et en humidité relative (%).

Tests concluants !

```
sketch_jan19a.ino
Non enregistré - sketch_jan19a.ino
1 #include <Wire.h>
2 #include "DFRobot_AHT20.h"
3 #include "Adafruit_CCS811.h"
4
5 #define PIN_LUMIERE 4
6
7 DFRobot_AHT20 capteurTempHum;
8 Adafruit_CCS811 capteurCO2;
9
10 float temperature = 0;
11 float humidite = 0;
12 float co2 = 0;
13 float voc = 0;
14 float lux = 0;
15 float ethylenePPM = 0;
16
17 void setup() {
18   Serial.begin(115200);
19 }
20
21 void loop() {
22   // Lecture des données du capteur AHT20
23   float tempHum = capteurTempHum.read();
24   temperature = tempHum.temperature;
25   humidite = tempHum.humidity;
26
27   // Lecture des données du capteur CCS811
28   float co2_ppm = capteurCO2.getCO2();
29   float voc_ppb = capteurCO2.getTVOC();
30   float lux = capteurCO2.getLight();
31   float ethylene_ppm = capteurCO2.getEthanol();
32
33   // Affichage des données
34   Serial.print("Temp : ");
35   Serial.print(temperature);
36   Serial.print(" C\n");
37   Serial.print("Humidite : ");
38   Serial.print(humidite);
39   Serial.print(" %\n");
40   Serial.print("Lumiere : ");
41   Serial.print(lux);
42   Serial.print(" Lux\n");
43   Serial.print("CO2 : ");
44   Serial.print(co2_ppm);
45   Serial.print(" ppm\n");
46   Serial.print("COV : ");
47   Serial.print(voc_ppb);
48   Serial.print(" ppb\n");
49   Serial.print("Ethylene : ");
50   Serial.print(ethylene_ppm);
51   Serial.print(" ppm\n");
52   Serial.println("-----");
53   delay(1000);
54 }
```

Sortie Moniteur série x

Message (Enter to send message to 'DFRobot FireBeetle 2 ESP32-C6' on 'COM5') Nouvelle ligne 115200 baud

```
--- MONITORING ---
Temp      : 21.08 C
Humidite  : 61.88 %
Lumiere   : 53.12 Lux
CO2       : 895.00 ppm
COV       : 75.00 ppb
Ethylene  : 0.00 ppm
-----
```